PROYECTO INTERMODULAR · 2.º DAM

AutoTrack

Plataforma Inteligente de Mantenimiento Predictivo y Gestión de Taller

---

Memoria técnica

Capítulo 1: Introducción · Capítulo 2: Antecedentes y estado del arte · Bibliografía

Equipo de desarrollo

Carmen Cerdán López — Product Owner · Backend & Security Developer

Andrea Zamora — Scrum Master · Backend & Database Developer

Carlos López — Frontend Desktop Developer (WPF/MVVM)

Brian Falcon — Frontend Mobile Developer (Android/Kotlin) & QA

Ciclo Formativo: Desarrollo de Aplicaciones Multiplataforma (DAM), 2.º curso

Módulo: Proyecto Intermodular · Bloque 2 (semanas 4 a 7)

Repositorio: github.com/CharlosLR/DAM2-Proyecto-AutoTrack

Tablero: github.com/users/CharlosLR/projects/3

Fecha: noviembre de 2026

Índice

Capítulo 1. Introducción 2
1.1. Presentación del problema y justificación 2
1.1.1. Contexto del sector 2
1.1.2. Problema detectado 2
1.1.3. Motivación y justificación del proyecto 2
1.2. Objetivos del proyecto 3
1.2.1. Objetivo general 3
1.2.2. Objetivos específicos (criterios SMART) 3
1.3. Descripción de la solución propuesta y propuesta de valor 4
1.4. Alcance y límites del sistema 5
1.5. Estructura de la memoria 6

Capítulo 2. Antecedentes y estado del arte 7
2.1. Revisión de sistemas existentes 7
2.1.1. Carsu (software comercial de gestión de taller) 7
2.1.2. Drivvo (aplicación comercial para conductores) 7
2.1.3. LubeLogger (proyecto de código abierto) 7
2.1.4. Método tradicional no automatizado 8
2.2. Análisis comparativo (benchmarking) 9
2.3. Justificación del stack tecnológico 10
2.3.1. Capa de presentación de escritorio: WPF, XAML y MVVM 10
2.3.2. Capa móvil: Kotlin y Jetpack Compose 10
2.3.3. Capa de backend y servicios: ASP.NET Core Web API 11
2.3.4. Capa de persistencia: PostgreSQL y Entity Framework Core 11
2.3.5. Librerías y servicios auxiliares 12

Declaración sobre el uso de inteligencia artificial generativa ..... ¡Error! Marcador no definido.

Bibliografía 13

Capítulo 1. Introducción

1.1. Presentación del problema y justificación

1.1.1. Contexto del sector

AutoTrack se sitúa en el sector de la automoción, en concreto en los servicios de posventa y en los talleres de reparación independientes, con la Comunidad Valenciana como ámbito inicial y el resto de España como mercado natural. Es un sector grande y envejecido. Según el informe de ANFAC e Ideauto, el parque español llegó en 2025 a 31,7 millones de vehículos y la edad media de los turismos subió hasta los **14,6 años** (14,5 en 2024). Además, el 28,9 % de los turismos, más de 7,75 millones de unidades, tiene más de 20 años [1].

Mantener ese parque es tarea de una red muy atomizada de pequeños negocios. El Libro Blanco del Taller de CONEPA, presentado en mayo de 2024, contabiliza **46.426 talleres** en España, 25.018 de ellos pertenecen a empresarios autónomos y 28.874 tienen solo uno o dos empleados (son grupos que se solapan). En 2023 el sector facturó de forma directa 15.553 millones de euros, un 4 % más que el año anterior [2]. Este perfil de taller pequeño e independiente, con poco personal administrativo, es el cliente al que se dirige la aplicación de escritorio de AutoTrack.

1.1.2. Problema detectado

Sobre un parque cada vez más antiguo, el mantenimiento se aplaza con frecuencia. Un estudio de OpinionWay para Aramis Group indica que el **36 %** de los conductores españoles ha pospuesto reparaciones o mantenimientos no urgentes por motivos económicos, seis puntos por encima de la media europea [3]. El ahorro es aparente: cambiar a tiempo el aceite y el filtro cuesta entre 70 y 120 euros y sustituir la correa de distribución, unos 400 euros de media, mientras que, si la correa se rompe, la reparación suele superar los 1.500 euros en muchos motores [4].

El problema de fondo es que el mantenimiento sigue un modelo **reactivo o de calendario fijo**. El conductor va al taller cuando la avería ya se ha producido o cuando vence un plazo genérico, como la revisión anual, sin tener en cuenta el desgaste real de cada pieza, que depende de los kilómetros recorridos y del tiempo transcurrido. Ni el conductor sabe qué pieza está cerca del final de su vida útil, ni el taller tiene esa información para anticiparse y planificar su carga de trabajo.

En el lado del taller, cuando las citas, las órdenes de reparación y los presupuestos se gestionan con agenda en papel, hojas de cálculo o mensajes, aparecen tres carencias concretas: listas de espera difíciles de ordenar y filtrar, poca trazabilidad de qué se hizo en cada vehículo y la imposibilidad de entregar al cliente un historial completo y un presupuesto detallado sin trabajo manual.

1.1.3. Motivación y justificación del proyecto

AutoTrack se justifica porque une dos partes que hoy funcionan por separado: el seguimiento del desgaste que necesita el conductor y la gestión diaria que necesita el taller. Como se detalla en el Capítulo 2, el software de gestión de taller se centra en la agenda, los presupuestos y la comunicación con el cliente, y las aplicaciones de conductor y las de código abierto registran mantenimientos y recordatorios sin conexión con el taller. Ninguna de las herramientas revisadas combina la gestión del taller con un cálculo del estado de cada pieza compartido con el conductor.

Un sistema que calcule el estado de cada pieza a partir del kilometraje real y comparta ese estado con el taller aporta beneficios tangibles frente a la situación actual:

- **Para el conductor:** sabe qué pieza revisar y cuándo, sin tener que recordarlo, lo que reduce el riesgo de que un mantenimiento barato aplazado acabe en una avería cara.
- **Para el taller:** recibe el vehículo antes de que la avería sea grave, organiza la agenda con filtros y deja registrada cada intervención.
- **Para ambos:** un único historial del vehículo, consultable desde el móvil y exportable en PDF junto con el presupuesto detallado.

1.2. Objetivos del proyecto

1.2.1. Objetivo general

Desarrollar, antes de la entrega del Bloque 4 del módulo, un sistema cliente-servidor formado por una aplicación de escritorio para el taller (WPF), una aplicación Android para el conductor y una API REST común que calcule el estado de desgaste de las piezas de cada vehículo a partir del kilometraje y del tiempo transcurrido, y que permita al taller gestionar clientes y vehículos, consultar la agenda de citas y el estado de las reparaciones, y entregar al cliente un informe PDF con el historial y el presupuesto.

1.2.2. Objetivos específicos (criterios SMART)

Cada objetivo específico describe un resultado verificable del sistema, no una tarea del tablero Kanban. Todos tienen como plazo la entrega del Bloque 4 (implementación) y se comprobarán con las pruebas que se documentarán en el Capítulo 5. Los códigos CU-01 a CU-05 son los casos de uso del producto mínimo viable (MVP) definidos en la Ficha de Propuesta, que se resumen en el apartado 1.4.

OE-01 (Seguridad y control de acceso): implementar en la API la autenticación con tokens JWT y el control de acceso por roles (administrador, mecánico y cliente) de forma que el 100 % de los endpoints que devuelven datos personales rechacen con código 401 o 403 las peticiones sin token válido o con un rol no autorizado, y que ninguna contraseña se almacene en texto plano (hash BCrypt), verificado con pruebas de integración automatizadas.

OE-02 (Cálculo del desgaste): implementar en el servidor el cálculo del estado de cada pieza (CU-03) como el mayor de dos porcentajes (kilómetros recorridos sobre su vida útil en kilómetros, y meses transcurridos sobre su vida útil en meses) y clasificarlo en verde (menos del 70 %), ámbar (del 70 % a menos del 90 %) o rojo (90 % o más), validado con al menos 20 casos de prueba unitarios, incluidos los valores límite de cada umbral, que se superen en su totalidad.

OE-03 (Persistencia transaccional): diseñar e implementar una base de datos relacional en PostgreSQL con Entity Framework Core y migraciones versionadas en el repositorio, de modo que el registro de una lectura de kilometraje y la actualización del estado de todas las piezas del vehículo (CU-03) se guarden de forma atómica (todo o nada), comprobado con una prueba que provoque un fallo a mitad de la operación y verifique que no queda ningún registro parcial.

OE-04 (Aplicación de escritorio e interfaz): desarrollar la aplicación del taller en WPF (.NET 10) con el patrón MVVM que permita el CRUD de clientes y vehículos con validación de campos (CU-02) y la consulta de la agenda con filtros combinados, ordenación y paginación (CU-04), mostrando

cada página de resultados en menos de 1 segundo con 10.000 citas de prueba y cumpliendo los criterios de WCAG 2.1 de contraste mínimo (4,5:1) y de manejo completo con teclado [5].

OE-05 (Aplicación móvil del conductor): desarrollar la aplicación Android en Kotlin con Jetpack Compose que muestre el semáforo de cada pieza y permita registrar una nueva lectura de kilometraje, de forma que el estado actualizado aparezca en pantalla en menos de 3 segundos tras guardar la lectura, medido en un dispositivo de pruebas con conexión wifi.

OE-06 (Informe PDF): generar desde la ficha del vehículo un informe PDF con el historial de mantenimiento y el presupuesto detallado (líneas, base imponible, IVA y total) en menos de 5 segundos para un vehículo con 50 intervenciones registradas (CU-05).

Tabla 1. Trazabilidad de los objetivos específicos

Objetivo	Caso de uso (MVP)	Módulo de 2.º DAM	Responsable principal
OE-01	CU-01	PSP (servicios y seguridad)	Carmen Cerdán López
OE-02	CU-03	PSP (lógica en el servidor)	Carmen Cerdán López
OE-03	CU-03 (persistencia)	AD (acceso a datos)	Andrea Zamora
OE-04	CU-02, CU-04	DI (interfaces)	Carlos López
OE-05	CU-03 (consulta)	PMDM (dispositivos móviles)	Brian Falcon
OE-06	CU-05	DI y AD	Carlos López

1.3. Descripción de la solución propuesta y propuesta de valor

AutoTrack es un sistema cliente-servidor. Las dos aplicaciones nunca acceden directamente a la base de datos: se comunican con una API REST que concentra la lógica, incluido el cálculo del desgaste, y que valida un token JWT en cada petición. Desde el punto de vista de cada tipo de usuario, el sistema funciona así:

- **Conductor, con rol cliente (aplicación Android):** inicia sesión, consulta sus vehículos y ve el estado de cada pieza con un semáforo verde, ámbar o rojo, junto con los kilómetros que le quedan y la fecha del último cambio. Cuando introduce el kilometraje actual, el servidor recalcula el estado. También puede consultar el historial de intervenciones que registra el taller.
- **Recepcionista o administrador del taller, con rol administrador (aplicación de escritorio):** da de alta clientes y vehículos con validación de datos, gestiona la agenda de citas con filtros por fecha, estado o mecánico, abre órdenes de reparación con su presupuesto y genera el informe PDF para el cliente.
- **Mecánico, con rol mecánico (aplicación de escritorio):** consulta las órdenes que tiene asignadas, actualiza su estado y registra las piezas sustituidas, lo que reinicia el contador de desgaste de esas piezas.

El término «mantenimiento predictivo» que aparece en el nombre del proyecto se usa en un sentido práctico: el cálculo sigue reglas deterministas a partir de la vida útil de cada pieza en kilómetros y en meses, no modelos de inteligencia artificial. Esta decisión hace que el resultado sea explicable al cliente y fácil de probar. La Tabla 2 resume qué procesos manuales sustituye el sistema.

Tabla 2. Procesos que AutoTrack automatiza frente al método tradicional

Proceso	Método tradicional	Con AutoTrack
Saber cuándo revisar	Recordatorio fijo («revisión cada año») o pegatina del último cambio de aceite	Estado de cada pieza calculado por kilómetros y meses, con semáforo en el móvil
Agenda del taller	Agenda en papel u hoja de cálculo	Agenda digital con filtros combinados, ordenación y paginación
Historial del vehículo	Fichas en papel repartidas entre carpetas y ordenadores	Historial único, compartido entre el taller y el conductor
Presupuesto e informe	Documento elaborado a mano en un procesador de textos	Informe PDF generado desde la ficha del vehículo

El valor diferencial frente a las soluciones existentes es la combinación de ambas partes en un mismo sistema: el taller y el conductor trabajan sobre los mismos datos del vehículo, y el aviso se basa en el uso real de cada pieza y no solo en una fecha.

1.4. Alcance y límites del sistema

El alcance se ha priorizado con la técnica MoSCoW, partiendo del MVP definido en la Ficha de Propuesta del Bloque 1. El criterio para decidir qué entra en el producto mínimo viable (MVP) es que el equipo pueda demostrar el flujo completo que justifica el proyecto: registrar un vehículo, calcular el estado de sus piezas, gestionar la cita y la orden en el taller y entregar el informe al cliente.

Tabla 3. Priorización MoSCoW del alcance

Prioridad	Funcionalidades
Must have (MVP)	CU-01 autenticación y control de acceso por roles con JWT. CU-02 CRUD de clientes y vehículos con validación. CU-03 cálculo automatizado del desgaste de piezas. CU-04 consulta de la agenda con filtros, ordenación y paginación. CU-05 informe PDF con historial y presupuesto, que incluye registrar la orden de reparación y su presupuesto.
Should have	Aviso de avería con foto o vídeo desde el móvil, notificaciones en tiempo real más allá de los avisos clave y aprobación de presupuestos desde el móvil.
Could have	Inventario de repuestos, facturación e integración con un sistema de gestión empresarial (Odoo).
Won't have (esta versión)	Aplicación para iOS, pagos en línea, lectura automática del kilometraje mediante OBD o telemetría y gestión de varios talleres o franquicias.

Las funciones *Should have* dependen de la sincronización en tiempo real entre el escritorio y el móvil, que en la Ficha de Propuesta se identificó como el principal riesgo técnico. Por eso el MVP se apoya en peticiones REST y limita el tiempo real a avisos de estado clave, como «tu coche está listo». Las funciones *Could have* pertenecen al ámbito de los sistemas de gestión empresarial y solo se abordarán si el MVP está terminado y probado.

Además, el sistema tiene estos límites conocidos:

- El kilometraje lo introduce el conductor o el taller; no se lee automáticamente del vehículo.

- La vida útil de cada pieza parte de valores de referencia que el taller puede configurar; no se integran datos de fabricantes.
- La aplicación de escritorio solo funciona en Windows, porque WPF no es multiplataforma [6]; la parte multiplataforma la cubren la API y la aplicación Android.

1.5. Estructura de la memoria

El resto de este documento se organiza de la siguiente manera:

- El **Capítulo 2** analiza las soluciones existentes y justifica el stack tecnológico elegido.
- El **Capítulo 3** detalla los requisitos funcionales y no funcionales y los casos de uso.
- El **Capítulo 4** describe la arquitectura, el modelo de datos y los prototipos de interfaz.
- El **Capítulo 5** expone la implementación, las pruebas realizadas y sus resultados.
- El **Capítulo 6** resume las conclusiones y el trabajo futuro.
- Los **Apéndices A y B** contienen el Manual de Usuario y la Guía de Despliegue Técnico.

Capítulo 2. Antecedentes y estado del arte

Antes de proponer una solución propia, el equipo ha revisado cómo se resuelve hoy el problema descrito en el Capítulo 1. Se han seleccionado cuatro referencias que cubren los dos lados del problema: un software comercial de gestión de taller, una aplicación comercial para conductores, un proyecto de código abierto y el método tradicional no automatizado. Los criterios de comparación se han elegido a partir de las necesidades del proyecto y de las características de calidad del modelo ISO/IEC 25010 [7], como la adecuación funcional, la eficiencia de desempeño, la compatibilidad y la seguridad. La información de cada producto procede de su documentación y sus páginas oficiales, consultadas el 1 de noviembre de 2026.

2.1. Revisión de sistemas existentes

2.1.1. Carsu (software comercial de gestión de taller)

- **Tipo:** Comercial, software como servicio (SaaS) con suscripción mensual o anual.
- **Plataforma:** Web, en la nube; no requiere instalación.
- **URL oficial:** https://www.carsu.com/es
- **Modelo de precio:** Plan Starter, 29,90 € al mes (2 usuarios y hasta 500 clientes); plan Pro, 59,90 € al mes (5 usuarios y hasta 2.500 clientes); plan Enterprise, a medida. Precios sin IVA, con un 17 % de descuento en pago anual y prueba gratuita [8].

Carsu es un software de gestión en la nube para talleres mecánicos, de motos y de neumáticos. Su punto fuerte es la comunicación con el cliente: integra WhatsApp Business, Viber, SMS y correo, envía recordatorios automáticos de revisión, ITV y neumáticos y ofrece un formulario de citas para la web del taller [9]; los presupuestos se envían por correo, SMS o WhatsApp [10]. Como limitaciones, la documentación no muestra una aplicación propia para el conductor ni un cálculo del desgaste por pieza, y el plan básico limita usuarios y clientes con un coste de 358,80 € al año sin IVA.

2.1.2. Drivvo (aplicación comercial para conductores)

- **Tipo:** Comercial con plan gratuito (freemium).
- **Plataforma:** Android, iOS y versión web.
- **URL oficial:** https://www.drivvo.com/en/
- **Modelo de precio:** Gratuito hasta 2 vehículos; Individual, 24,90 $ al año; PRO, 49,90 $ al año o 4,90 $ al mes; Flota, 42 $ por vehículo y año [11].

Drivvo es una aplicación para particulares y flotas que registra repostajes, gastos y mantenimientos con fecha, odómetro y lugar, y avisa de las próximas tareas por fecha o por kilometraje [12]; los informes se exportan a CSV o Excel solo en los planes de pago [11]. Es la referencia más cercana a la parte del conductor de AutoTrack, pero su documentación no describe ninguna conexión con el taller: el historial lo introduce el usuario y los avisos dependen de intervalos que configura él mismo.

2.1.3. LubeLogger (proyecto de código abierto)

- **Tipo:** Código abierto, licencia MIT.

- **Plataforma:** Web autoalojada, distribuida como imagen Docker y como ejecutable para Windows.
- **URL / repositorio:** https://github.com/hargata/lubelog
- **Modelo de precio:** Gratuito; el coste es el del servidor donde se instale.

LubeLogger es un gestor web autoalojado de mantenimiento y consumo de vehículos, publicado por Hargata Softworks con licencia MIT [13], con recordatorios por fecha, por odómetro o por lo que ocurra primero, varios usuarios, exportación CSV y API [14]. Es un proyecto .NET que usa LiteDB por defecto y recomienda PostgreSQL para más escala [15], lo que confirma que ese stack encaja en el dominio. Está pensado para el propietario particular: no incluye citas, órdenes ni presupuestos, no tiene aplicación móvil nativa y requiere mantener un servidor propio.

2.1.4. Método tradicional no automatizado

- **Tipo:** Método manual y herramientas genéricas: agenda en papel, fichas por vehículo, hoja de cálculo y mensajería.
- **Plataforma:** Papel y ofimática de escritorio.
- **Modelo de precio:** Sin licencia específica; material de oficina o software ofimático ya disponible.

Es el punto de partida que AutoTrack pretende sustituir. El taller anota las citas en una agenda, abre una ficha o una pestaña de hoja de cálculo por vehículo y avisa al cliente por teléfono o mensajería, y el conductor se guía por el libro de mantenimiento o por la pegatina del último cambio de aceite. No tiene coste de licencia, pero no calcula nada: la próxima revisión depende de que alguien la recuerde, la información queda repartida entre papel, ordenadores y móviles personales, y no hay copia de seguridad ni registro de quién modificó cada dato. Tampoco permite consultas con filtros ni genera informes sin trabajo manual.

2.2. Análisis comparativo (benchmarking)

La Tabla 4 compara las cuatro referencias con la propuesta de AutoTrack en diez criterios. Cuando un dato no aparece en la documentación oficial del producto se indica como «no documentado», en lugar de suponerlo.

Tabla 4. Matriz de benchmarking

Criterio	Carsu (comercial, taller)	Drivvo (comercial, conductor)	LubeLogger (código abierto)	Método manual	AutoTrack (propuesta)
Plataforma	Web en la nube	Android, iOS y web	Web autoalojada (Docker o Windows)	Papel y hoja de cálculo	Escritorio WPF (taller), Android (conductor) y API REST
Coste de licencia	Desde 29,90 €/mes sin IVA	Gratuito hasta 2 vehículos; desde 24,90 $/año	Gratuito (MIT)	Sin licencia	Sin coste de licencia para el equipo: .NET, PostgreSQL y librerías gratuitas (QuestPDF con licencia comunitaria)
Rendimiento y fluidez	Depende de la conexión a internet	No documentado	Depende del servidor; LiteDB por defecto, PostgreSQL para más escala	Búsquedas manuales, lentas con muchos registros	Objetivo: < 1 s por página con 10.000 citas (OE-04)
Persistencia transaccional (ACID)	No documentado	No documentado	LiteDB (fichero) o PostgreSQL opcional	No	Sí: PostgreSQL con EF Core (OE-03)
Accesibilidad (WCAG 2.1)	No documentado	No documentado	No documentado	No aplica	Contraste 4,5:1 y manejo con teclado (OE-04)
Generación de informes PDF	Informes exportables; formato no especificado	Informes y gráficos; exportación CSV/Excel en planes de pago	Informes de vehículo y exportación CSV	Manual (procesador de textos)	PDF con historial y presupuesto (CU-05)
Integración con otros sistemas	WhatsApp Business, Viber, SMS y correo	Exportación CSV/Excel	API, inicio de sesión único (SSO) e importación CSV	Ninguna	API REST con JWT; Odoo como *Could have*
Soporte y comunidad	Chat y teléfono	Soporte prioritario en planes de pago	Comunidad en GitHub (más de 2.800 estrellas)	No aplica	Equipo del proyecto y repositorio público
Cálculo del estado por uso	Recordatorios de revisión, ITV y neumáticos; sin cálculo por pieza documentado	Avisos por fecha o kilometraje configurados por el usuario	Recordatorios por fecha, odómetro o lo que ocurra primero	No	Semáforo por pieza según kilómetros y meses (CU-03)
Conexión taller-conductor	Mensajería; sin aplicación para el conductor	No	No (orientado al propietario)	Teléfono o mensajería	Historial compartido entre escritorio y móvil a través de la API

La matriz muestra que cada herramienta resuelve bien una parte del problema. Carsu es la más completa en la gestión del taller y en la comunicación con el cliente, Drivvo y LubeLogger cubren el seguimiento del mantenimiento desde el lado del propietario, y el método manual no tiene coste pero tampoco automatiza nada. Ninguna combina la gestión del taller con un cálculo del estado de cada pieza compartido con el conductor, que es el hueco que ocupa AutoTrack. Al mismo tiempo, la comparación señala dos puntos en los que la propuesta parte con desventaja y que el equipo asume: LubeLogger tiene una comunidad activa en GitHub y Drivvo declara más de cinco millones de descargas [12], mientras que AutoTrack parte de cero; además, la aplicación de escritorio de AutoTrack solo funcionará en Windows.

2.3. Justificación del stack tecnológico

Todo el sistema, salvo la aplicación móvil, se desarrolla sobre **.NET 10**, que Microsoft publicó el 11 de noviembre de 2025 como versión de soporte a largo plazo (LTS), con soporte hasta el 14 de noviembre de 2028 [16]. Ese plazo cubre todo el desarrollo del proyecto y su posible uso posterior, y permite compartir el lenguaje C# y las clases del modelo entre la aplicación de escritorio y la API.

2.3.1. Capa de presentación de escritorio: WPF, XAML y MVVM

La aplicación del taller se construye con Windows Presentation Foundation (WPF). Su principal ventaja para este proyecto es el enlace de datos: los elementos de la interfaz enlazados reflejan automáticamente los cambios del origen de datos cuando este notifica que ha cambiado [17]. Esto encaja con el patrón Model-View-ViewModel (MVVM), que separa la presentación de la lógica de dominio [18] y facilita probar la lógica sin abrir ventanas. WPF añade además la definición declarativa de la interfaz en XAML, estilos y plantillas reutilizables y un sistema de comandos y validación pensado para MVVM [19].

Frente a las alternativas:

- **Windows Forms** también es un marco para aplicaciones de escritorio de Windows, con enlace de datos y un diseñador visual de arrastrar y colocar [20], pero no ofrece la definición declarativa en XAML ni el sistema de estilos y plantillas de WPF [19], que el proyecto necesita para dar un aspecto homogéneo a la agenda, los formularios y las fichas.
- **Electron** permite crear aplicaciones de escritorio con JavaScript, HTML y CSS integrando Chromium y Node.js en su binario [21]. Obligaría a introducir un segundo lenguaje y ecosistema en un proyecto cuyo backend es C#, y cada aplicación incluiría su propio motor de navegador.

La limitación asumida es que WPF solo se ejecuta en Windows [6]. Es aceptable porque la aplicación de escritorio está pensada para el puesto de recepción del taller, y la parte multiplataforma la cubren la API y la aplicación Android.

2.3.2. Capa móvil: Kotlin y Jetpack Compose

La aplicación del conductor se desarrolla en Kotlin con Jetpack Compose. Google se ha comprometido a que el desarrollo para Android sea cada vez más *Kotlin-first* y recomienda empezar con Kotlin las nuevas aplicaciones [22], y presenta Jetpack Compose como su kit moderno recomendado para crear interfaces nativas [23]. Como referencia del lenguaje, el equipo utiliza la segunda edición de *Kotlin in Action* [24].

La alternativa valorada fue **Flutter**, cuyas aplicaciones se escriben en Dart y se dibujan con su propio motor de renderizado, lo que permite publicar en varias plataformas desde un solo código [25]. Esa ventaja tiene poco peso en este proyecto porque la versión para iOS está fuera del alcance (apartado 1.4), y elegir Flutter supondría añadir un lenguaje más.

2.3.3. Capa de backend y servicios: ASP.NET Core Web API

La API se implementa con ASP.NET Core Web API. Con el atributo ApiController aplicado a los controladores, los errores de validación del modelo devuelven automáticamente una respuesta HTTP 400 [26], lo que unifica el tratamiento de datos incorrectos que llegan desde el escritorio y desde el móvil. Al compartir C# con la aplicación WPF, las clases de transferencia de datos y sus reglas de validación se escriben una sola vez [27].

La seguridad se basa en JSON Web Token, un formato estándar y compacto para transmitir afirmaciones (*claims*) entre dos partes, que pueden ir firmadas o cifradas [28]. ASP.NET Core incluye la autenticación JWT de tipo *bearer*, y su documentación insiste en que la API debe validar por completo la firma, el emisor, la audiencia y la caducidad de cada token [29]. Esa validación es la base del control por roles del objetivo OE-01.

El canal de avisos en tiempo real se basa en el protocolo WebSocket, que mantiene un canal persistente y bidireccional sobre una conexión TCP [30] y que ASP.NET Core admite de forma nativa [31]. Para no gestionar las conexiones a bajo nivel, en el diseño (Capítulo 4) se valorará implementarlo con SignalR, una biblioteca que añade funciones en tiempo real y usa WebSockets como transporte preferido [32]. En el MVP su uso se limita a los avisos de estado clave, tal como se decidió para mitigar el riesgo de sincronización.

Sobre la compilación anticipada (AOT nativo), la documentación indica que puede reducir el tiempo de arranque y el consumo de memoria, pero que MVC todavía no es compatible y que las API mínimas solo lo son parcialmente [33]. Como AutoTrack usa controladores, el MVP no dependerá de AOT nativo; se priorizan la claridad del código y la compatibilidad con EF Core.

Frente a **Node.js con Express**, ASP.NET Core evita introducir JavaScript en el servidor y permite usar Entity Framework Core y las mismas clases de C# en el escritorio y en la API.

2.3.4. Capa de persistencia: PostgreSQL y Entity Framework Core

La base de datos es **PostgreSQL**, un sistema objeto-relacional de código abierto que cumple ACID desde 2001 [34]. Su control de concurrencia multiversión (MVCC) hace que las lecturas y las escrituras no se bloqueen entre sí [35], algo útil cuando el taller y los conductores consultan y actualizan datos a la vez. Los datos de AutoTrack son muy relacionales (clientes, vehículos, piezas, lecturas de kilometraje, citas, órdenes, presupuestos y líneas), y las claves foráneas y las transacciones garantizan su integridad.

El acceso a datos se hace con **Entity Framework Core**, un ORM de código abierto y multiplataforma [36], a través del proveedor Npgsql, que se comporta como cualquier otro proveedor de EF Core [37]. EF Core genera las migraciones comparando el modelo actual con una instantánea del anterior [38], de modo que los cambios del esquema quedan versionados en Git junto con el código. Las consultas se escriben en LINQ con tipado fuerte, y cuando hace falta SQL propio, los métodos FromSql y FromSqlInterpolated están protegidos contra la inyección de SQL, mientras que FromSqlRaw exige

cuidado porque puede ser vulnerable si se usa mal [39]. El equipo sigue *Entity Framework Core in Action* como guía de buenas prácticas [40].

Frente a las alternativas:

- **ADO.NET** obligaría a escribir a mano el SQL y la conversión entre filas y objetos; EF Core permite trabajar con la base de datos mediante objetos .NET y elimina la mayor parte de ese código de acceso a datos [36], además de aportar las migraciones.
- **MongoDB** admite transacciones ACID de varios documentos, con garantías que dependen de la configuración de lectura y escritura [41], pero un modelo documental encaja peor con datos tan relacionados como los de un taller.

2.3.5. Librerías y servicios auxiliares

- **CommunityToolkit.Mvvm:** kit de Microsoft para aplicar MVVM con menos código repetitivo; se distribuye para .NET Standard y se puede usar con WPF [42].
- **QuestPDF:** librería para generar documentos PDF con una API fluida en C#, usada para el informe del CU-05. Es gratuita para particulares, organizaciones sin ánimo de lucro, proyectos de código abierto y empresas con menos de 1 millón de dólares de ingresos anuales; por encima de esa cifra requiere licencia comercial [43].
- **BCrypt.Net-Next:** adaptación a .NET de jBCrypt, con licencia MIT, que aplica un hash basado en Blowfish con un factor de trabajo configurable [44]. Se usa para almacenar las contraseñas (OE-01).

Bibliografía

[1] J. C. Payo, «Alerta en seguridad y emisiones: la edad media del coche español ya roza los 15 años», *Autofácil*, 27-feb-2026. [En línea]. Disponible en: https://www.autofacil.es/alerta-en-seguridad-y-emisiones-la-edad-media-del-coche-espanol-ya-roza-los-15-anos/. [Accedido: 01-nov-2026].

[2] IFEMA MADRID, «MOTORTEC MADRID acogió la presentación del Libro Blanco del Taller de CONEPA», *IFEMA MADRID: Motortec*, 27-may-2024. [En línea]. Disponible en: https://www.ifema.es/motortec/noticias/presentacion-libro-blanco-conepa. [Accedido: 01-nov-2026].

[3] D. González, «El 36% pospone el mantenimiento del coche por dinero: qué se puede aplazar y qué acaba en avería cara», *Motor16*, 17-sep-2026. [En línea]. Disponible en: https://www.motor16.com/las-ultimas-noticias/posponer-mantenimiento-coche-averia-cara/. [Accedido: 01-nov-2026].

[4] D. González, «El 36 % de los conductores ha pospuesto el mantenimiento del coche: la revisión barata acaba en avería de 1.500 euros», *Motor16*, 14-sep-2026. [En línea]. Disponible en: https://www.motor16.com/las-ultimas-noticias/pospuesto-mantenimiento-coche-averia-1500-euros/. [Accedido: 01-nov-2026].

[5] W3C Accessibility Guidelines Working Group, «Web Content Accessibility Guidelines (WCAG) 2.1», *W3C Recommendation*, 5-jun-2018 (rev. 6-may-2025). [En línea]. Disponible en: https://www.w3.org/TR/WCAG21/. [Accedido: 01-nov-2026].

[6] Microsoft, «What is Windows Presentation Foundation - WPF», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/. [Accedido: 01-nov-2026].

[7] *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*, 2.ª ed. Ginebra, Suiza: ISO/IEC, 2023. [En línea]. Disponible en: https://www.iso.org/standard/78176.html. [Accedido: 01-nov-2026].

[8] Carsu Technologies, «Pricing», *carsu.com*, 2026. [En línea]. Disponible en: https://www.carsu.com/en/pricing. [Accedido: 01-nov-2026].

[9] Carsu Technologies, «Features», *carsu.com*, 2026. [En línea]. Disponible en: https://www.carsu.com/en/features. [Accedido: 01-nov-2026].

[10] Carsu Technologies, «Carsu [página principal, versión en español]», *carsu.com*, 2026. [En línea]. Disponible en: https://www.carsu.com/es. [Accedido: 01-nov-2026].

[11] Drivvo, «Pricing», *drivvo.com*, 2026. [En línea]. Disponible en: https://www.drivvo.com/en/pricing/. [Accedido: 01-nov-2026].

[12] Drivvo, «Drivvo [página principal]», *drivvo.com*, 2026. [En línea]. Disponible en: https://www.drivvo.com/en/. [Accedido: 01-nov-2026].

[13] Hargata Softworks, «hargata/lubelog», *GitHub*, 2026. [En línea]. Disponible en: https://github.com/hargata/lubelog. [Accedido: 01-nov-2026].

[14] Hargata Softworks, «LubeLogger», *lubelogger.com*, 2026. [En línea]. Disponible en: https://lubelogger.com/. [Accedido: 01-nov-2026].

[15] Hargata Softworks, «Technical Docs», *LubeLogger Documentation*, 2026. [En línea]. Disponible en: https://docs.lubelogger.com/Advanced/Technical%20Docs. [Accedido: 01-nov-2026].

[16] Microsoft, «.NET Support Policy», *dotnet.microsoft.com*, 2026. [En línea]. Disponible en: https://dotnet.microsoft.com/en-us/platform/support/policy. [Accedido: 01-nov-2026].

[17] Microsoft, «Información general sobre la vinculación de datos - WPF», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/es-es/dotnet/desktop/wpf/data/. [Accedido: 01-nov-2026].

[18] M. Fowler, «GUI Architectures», *martinfowler.com*, 18-jul-2006. [En línea]. Disponible en: https://martinfowler.com/eaaDev/uiArchs.html. [Accedido: 01-nov-2026].

[19] A. Troelsen y P. Japikse, *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming*, 11.ª ed. Berkeley, CA, EE. UU.: Apress, 2022. ISBN: 978-1-4842-7868-0.

[20] Microsoft, «Descripción general de Windows Forms», *Microsoft Learn*, 2025. [En línea]. Disponible en: https://learn.microsoft.com/es-es/dotnet/desktop/winforms/overview/. [Accedido: 01-nov-2026].

[21] OpenJS Foundation y colaboradores de Electron, «What is Electron?», *Electron Documentation*, 2026. [En línea]. Disponible en: https://www.electronjs.org/docs/latest/. [Accedido: 01-nov-2026].

[22] Android Developers, «Android's Kotlin-first approach», *developer.android.com*, 2026. [En línea]. Disponible en: https://developer.android.com/kotlin/first. [Accedido: 01-nov-2026].

[23] Android Developers, «Jetpack Compose UI App Development Toolkit», *developer.android.com*, 2026. [En línea]. Disponible en: https://developer.android.com/compose. [Accedido: 01-nov-2026].

[24] S. Aigner, R. Elizarov, S. Isakova y D. Jemerov, *Kotlin in Action*, 2.ª ed. Shelter Island, NY, EE. UU.: Manning, 2024. ISBN: 978-1-61729-960-5.

[25] Flutter, «Flutter architectural overview», *Flutter Documentation*, 2026. [En línea]. Disponible en: https://docs.flutter.dev/resources/architectural-overview. [Accedido: 01-nov-2026].

[26] Microsoft, «Creación de API web con ASP.NET Core», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/es-es/aspnet/core/web-api/. [Accedido: 01-nov-2026].

[27] A. Lock, *ASP.NET Core in Action*, 3.ª ed. Shelter Island, NY, EE. UU.: Manning, 2023. ISBN: 978-1-63343-862-0.

[28] M. Jones, J. Bradley y N. Sakimura, «JSON Web Token (JWT)», IETF, RFC 7519, may-2015. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc7519. [Accedido: 01-nov-2026].

[29] D. Bowden, «Configurar la autenticación JWT de tipo bearer en ASP.NET Core», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/es-es/aspnet/core/security/authentication/configure-jwt-bearer-authentication. [Accedido: 01-nov-2026].

[30] I. Fette y A. Melnikov, «The WebSocket Protocol», IETF, RFC 6455, dic-2011. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc6455. [Accedido: 01-nov-2026].

[31] Microsoft, «Compatibilidad con WebSockets en ASP.NET Core», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/es-es/aspnet/core/fundamentals/websockets?view=aspnetcore-10.0. [Accedido: 01-nov-2026].

[32] Microsoft, «Información general sobre ASP.NET Core SignalR», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/es-es/aspnet/core/signalr/introduction. [Accedido: 01-nov-2026].

[33] Microsoft, «Compatibilidad de ASP.NET Core con AOT nativo», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/es-es/aspnet/core/fundamentals/native-aot. [Accedido: 01-nov-2026].

[34] The PostgreSQL Global Development Group, «About», *postgresql.org*, 2026. [En línea]. Disponible en: https://www.postgresql.org/about/. [Accedido: 01-nov-2026].

[35] The PostgreSQL Global Development Group, «13.1. Introduction», *PostgreSQL 18 Documentation*, 2026. [En línea]. Disponible en: https://www.postgresql.org/docs/current/mvcc-intro.html. [Accedido: 01-nov-2026].

[36] Microsoft, «Información general de Entity Framework Core», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/es-es/ef/core/. [Accedido: 01-nov-2026].

[37] The Npgsql Development Team, «Npgsql Entity Framework Core Provider», *Npgsql Documentation*, 2026. [En línea]. Disponible en: https://www.npgsql.org/efcore/. [Accedido: 01-nov-2026].

[38] Microsoft, «Descripción general de las migraciones - EF Core», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/es-es/ef/core/managing-schemas/migrations/. [Accedido: 01-nov-2026].

[39] Microsoft, «Consultas SQL - EF Core», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/es-es/ef/core/querying/sql-queries. [Accedido: 01-nov-2026].

[40] J. P. Smith, *Entity Framework Core in Action*, 2.ª ed. Shelter Island, NY, EE. UU.: Manning, 2021. ISBN: 978-1-61729-836-3.

[41] MongoDB, Inc., «Transactions», *MongoDB Database Manual*, 2026. [En línea]. Disponible en: https://www.mongodb.com/docs/manual/core/transactions/. [Accedido: 01-nov-2026].

[42] Microsoft, «Introducción al kit de herramientas MVVM», *Microsoft Learn*, 2026. [En línea]. Disponible en: https://learn.microsoft.com/es-es/dotnet/communitytoolkit/mvvm/. [Accedido: 01-nov-2026].

[43] QuestPDF, «QuestPDF/QuestPDF», *GitHub*, 2026. [En línea]. Disponible en: https://github.com/QuestPDF/QuestPDF. [Accedido: 01-nov-2026].

[44] BcryptNet, «BcryptNet/bcrypt.net: BCrypt.Net - Bringing updates to the original bcrypt package», *GitHub*, 2026. [En línea]. Disponible en: https://github.com/BcryptNet/bcrypt.net. [Accedido: 01-nov-2026].